Sample of Analyzing Stock Data

In [1]:
```
# Importing libraries for use during the analysis

from math import log, sqrt
import numpy as np
```

Prepping analysis by creating helper functions to be used during the analysis

In [2]:
```
# creating a function to calculate the log return

def calculate_log_return(start_price, end_price):
    return log(end_price / start_price)
```

In [3]:
```
# creating a function to calculate variance

def calculate_variance(dataset):
    mean = sum(dataset) / len(dataset)
    numerator = 0

    for data in dataset:
        numerator += (data - mean) ** 2
    return numerator / len(dataset)
```

In [4]:
```
# creating a function to calculate the standard deviation

def calculate_stddev(dataset):
    variance = calculate_variance(dataset)
    return sqrt(variance)
```

In [5]:
```
# creating a function to calculate the correlation coefficient

def calculate_correlation(set_x, set_y):
    sum_x = sum(set_x)
    sum_y = sum(set_y)
    sum_x2 = sum([x ** 2 for x in set_x])
    sum_y2 = sum([y ** 2 for y in set_y])
    sum_xy = sum([x * y for x, y in zip(set_x, set_y)])
    n = len(set_x)
    numerator = n * sum_xy - sum_x * sum_y
    denominator = sqrt((n * sum_x2 - sum_x ** 2) * (n * sum_y2 - sum_y ** 2))

    return numerator / denominator
```

In [6]:
```
# creating a conversion function to covert a number to be properly displayed as a percentage

def display_as_percentage(val):
    return '{:.1f}%'.format(val * 100)
```

Creating sample (fictional) data for analysis

Prices in each dataset are for each month from June 2018 through June 2019 (inclusive).

In [7]:
```
# Sample data from June 2018 through June 2019

amazon_prices = [1699.8, 1777.44, 2012.71, 2003.0, 1598.01, 1690.17, 1501.97, 1718.73, 1639.83, 1780.7
ebay_prices = [35.98, 33.2, 34.35, 32.77, 28.81, 29.62, 27.86, 33.39, 37.01, 37.0, 38.6, 35.93, 39.5]
```

Analyzing the two datasets

In [8]:
```
# calculating the log rate of return using a for loop
```

```
def get_returns(prices):
    returns = []
    for i in range(len(prices) - 1):
        start_price = prices[i]
        end_price = prices[i + 1]
        returns.append(calculate_log_return(start_price, end_price))
    return returns
```

A for loop was set up to read through all of the prices. The range(len(prices) - 1) calculation is used to prevent the loop from trying to read prices beyond the length of what is in the dataset. i + 1 is used to identify the end price, which is the starting price of the next month. The starting price and end price is used to calcuate the log rate of return and then appending it to the returns list.

In [9]:
```
# calculating the log returns for the amazon dataset

amazon_returns = get_returns(amazon_prices)
print(amazon_returns)
```

```
[0.044663529768886545, 0.12430794584153733, -0.004836016009131401, -0.22588695153690044, 0.056070010445
170376, -0.11805153581831997, 0.13480806622777397, -0.046993068074800755, 0.082442045949722, 0.07868064
267475429, -0.0818754077815861, 0.06465576316168306]
```

In [10]:
```
# calculating the log returns of the ebay dataset

ebay_returns = get_returns(ebay_prices)
print(ebay_returns)
```

```
[-0.080413352599944, 0.034052142745915476, -0.04708855595763511, -0.1287909136142863, 0.027727259743215
74, -0.061257840487993175, 0.18106448560390354, 0.10293169244250136, -0.00027023375384007574, 0.0423343
63826560736, -0.07167967534535787, 0.09472807078164892]
```

The two sets of results above display the log rate of return for the stock prices, unformatted.

In [20]:
```
print('The monthly log rates of return for Amazon are:',', '.join([display_as_percentage(rate) for rat
```

```
The monthly log rates of return for Amazon are: 4.5%, 12.4%, -0.5%, -22.6%, 5.6%, -11.8%, 13.5%, -4.7%,
8.2%, 7.9%, -8.2%, 6.5%
```

In [12]:
```
print('The monthly log rates of return for eBay are:',', '.join([display_as_percentage(rate) for rate
```

```
The monthly log rates of return for eBay are: -8.0%, 3.4%, -4.7%, -12.9%, 2.8%, -6.1%, 18.1%, 10.3%, -
0.0%, 4.2%, -7.2%, 9.5%
```

Both lines of code above format the monthly log rate of returns to an easier readable format. This also makes comparing the returns for both stock prices easier.

In [13]:
```
amazon_yearly = display_as_percentage(sum(amazon_returns))
print(amazon_yearly)
```

```
10.8%
```

In [14]:
```
ebay_yearly = display_as_percentage(sum(ebay_returns))
print(ebay_yearly)
```

```
9.3%
```

Both lines of code above return the yearly log rate of return for each respective stock. Amazon has a slightly higher return than eBay.

Assessing Investment Risk

In [15]:
```
amazon_variance = calculate_variance(amazon_returns)
print(amazon_variance)
```

```
0.010738060556609724
```

In [16]:
```
ebay_variance = calculate_variance(ebay_returns)
print(ebay_variance)
```

```
0.007459046435081462
```

The two lines of code above identifies the variance for each stock. Keep in mind, a higher variance signifies a riskier investment. However, it is not in the same unit of measurement as the original dataset, so it makes it a little more difficult to interpret.

In [17]:
```
amazon_stddev = display_as_percentage(calculate_stddev(amazon_returns))
print(amazon_stddev)
```

```
10.4%
```

In [18]:
```
ebay_stddev = display_as_percentage(calculate_stddev(ebay_returns))
print(ebay_stddev)
```

```
8.6%
```

The two lines of code above calculate the standard deviation for each stock. This metric is much easier to interpret since it is in the same unit of measurement as the original dataset.

In [19]:
```
amazon_ebay_corr = calculate_correlation(amazon_returns, ebay_returns)
print(amazon_ebay_corr)
```

```
0.6776978564073072
```

The above line of code calculates the correlation between the two stocks. This value indicates how closely the two stocks are related. Since both companies are online retail stores, there is a moderately strong relationship between each other. Meaning both companies are generally impacted by similar external forces (economic).